

TEAM X
AI Hackathon Celestial Quest

1) Preprocessing:

Preparing the dataset: adding a path column which contains the path of every image.

```
[26]: img_path='Train-20241112T104824Z-001/Train/'
[27]: df_train['img_path']=img_path+df_train['path']
[28]: df_train.head()
[28]:
```

	path	label	img_path
0	00001.jpg	22	Train-20241112T104824Z-001/Train/00001.jpg
1	00002.jpg	18	Train-20241112T104824Z-001/Train/00002.jpg
2	00003.jpg	40	Train-20241112T104824Z-001/Train/00003.jpg
3	00004.jpg	25	Train-20241112T104824Z-001/Train/00004.jpg
4	00005.jpg	5	Train-20241112T104824Z-001/Train/00005.jpg

```
[29]: df_new=pd.concat([df_train['label'],df_train['img_path']],axis=1)
[30]: df_new.head()
[30]:
```

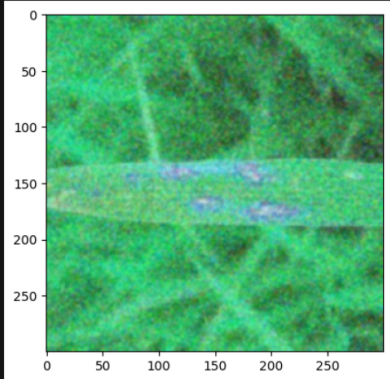
	label	img_path
0	22	Train-20241112T104824Z-001/Train/00001.jpg
1	18	Train-20241112T104824Z-001/Train/00002.jpg
2	40	Train-20241112T104824Z-001/Train/00003.jpg
3	25	Train-20241112T104824Z-001/Train/00004.jpg
4	5	Train-20241112T104824Z-001/Train/00005.jpg

```
[31]: df_dict=df_new.to_dict(orient='index')
```

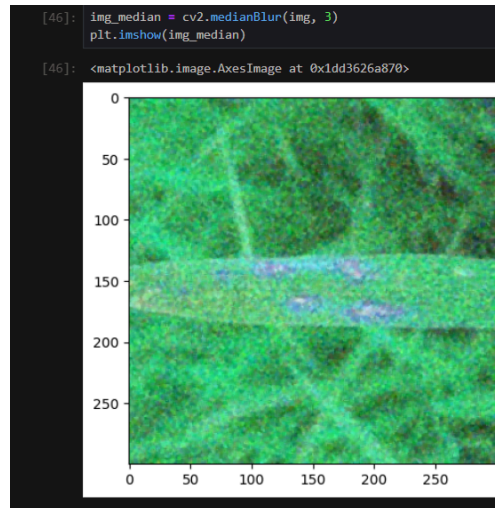
Testing filters: tested images after applying multiple filters:

- Gaussian filter:

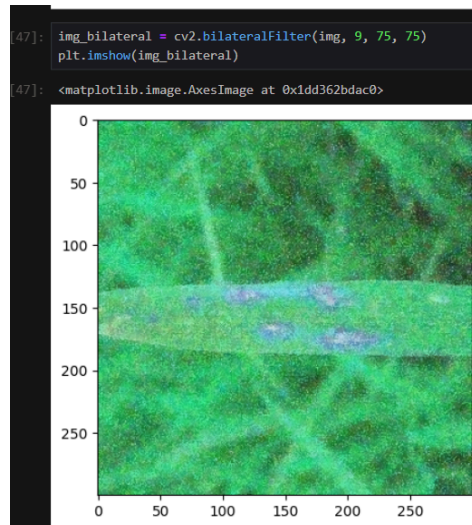
```
[45]: img_gaussian = cv2.GaussianBlur(img, (5, 5), 0)
      plt.imshow(img_gaussian)
[45]: <matplotlib.image.AxesImage at 0x1dd361d3c50>
```



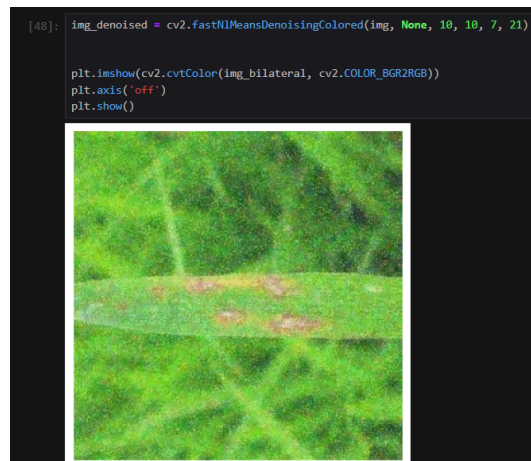
- Median filter:



- Bilateral filter:



- BGR to RGB conversion:



Removing Noices and distortion : Tested 2 different preprocess methods .

1). we have converted images from BRG format to RGB format, applied Pca for 0.5 components and applied Bilateral filtering to remove noises.

```
[50]: def preprocess_image(img_path, target_size=(300, 300)):
    img = cv2.imread(img_path)
    #Conversion BGR to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    # Resize the image to the target size
    img_resized = cv2.resize(img_rgb, target_size)
    # Denoise using Bilateral Filter
    img_bilateral = cv2.bilateralFilter(img_resized, 9, 75, 75)
    clahe = cv2.createCLAHE(cleptlimit=1.5, tileGridSize=(5,5))
    img_lab = cv2.cvtColor(img_bilateral, cv2.COLOR_RGB2LAB)
    img_lab[:, :, 0] = clahe.apply(img_lab[:, :, 0])
    img_clahe = cv2.cvtColor(img_lab, cv2.COLOR_LAB2RGB)

    img_normalized = img_clahe / 255.0
    return img_normalized

# Example usage
# Replace with your image path
img = cv2.imread(df_train['img_path'][4])
processed_img = preprocess_image(df_train['img_path'][4])
# Display the original and processed images side by side
plt.figure(figsize=(10, 6)) # Increase the figure size for larger display
#Original Image
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis('off')

# Processed image
plt.subplot(1, 2, 2)
plt.imshow(processed_img)
plt.title("Processed Image")
plt.axis('off')
```

[50]: (np.float64(-0.5), np.float64(149.5), np.float64(149.5), np.float64(-0.5))

Original Image



Processed Image



2) converting image from brg to Rgb and applying bilateral filter with edge preserving denoising and unsharp masking

from sharpening.

```
# Load the image

def prepro_img(img_path, target_size=(300, 300)):
    img = cv2.imread(img_path)
    # Convert to RGB (OpenCV uses BGR by default)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img_rgb, target_size)
    # Apply Bilateral Filter for edge-preserving denoising
    denoised_image = cv2.bilateralFilter(img_resized, d=9, sigmaColor=75, sigmaSpace=75)

    # Apply sharpening using Unsharp Masking
    blurred = cv2.GaussianBlur(denoised_image, (5, 5), 1.5)
    sharpened_image = cv2.addWeighted(denoised_image, 1.5, blurred, -0.5, 0)
    img_normalized = sharpened_image / 255.0
    return img_normalized

# Display the results
plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 1)
plt.imshow(cv2.imread(df_train['img_path'][4]))
plt.title('Original Image')
plt.axis('off')

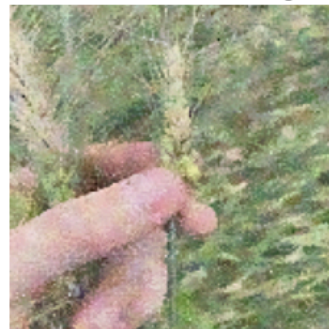
plt.subplot(1, 3, 2)
plt.imshow(prepro_img(df_train['img_path'][4], (150, 150)))
plt.title('Bilateral Filter Denoising')
plt.axis('off')

plt.show()
```

Original Image



Bilateral Filter Denoising



2) Augmentation:

Data augmentation is done to increase the number of images in the particular labels(example label 11 has 3 images etc) using the following

techniques .

```
1: # CustomDataGenerator inherits from Sequence
class CustomDataGenerator(Sequence):
    def __init__(self, df, batch_size, num_classes, target_size=(150,150), shuffle=True, augment=False):
        self.df = df
        self.batch_size = batch_size
        self.num_classes = num_classes
        self.target_size = target_size
        self.shuffle = shuffle
        self.augment = augment
        self.indices = np.arange(len(df))

        # ImageDataGenerator for real-time augmentation
        if self.augment:
            self.datagen = ImageDataGenerator(
                rotation_range=15,
                width_shift_range=0.1,
                height_shift_range=0.1,
                shear_range=0.1,
                zoom_range=0.1,
                horizontal_flip=True,
                fill_mode="nearest"
            )

        self.on_epoch_end()

    def __len__(self):
        # Number of batches per epoch
        return int(np.ceil(len(self.df) / self.batch_size))

    def __getitem__(self, index):
        # Generate one batch of data
        start = index * self.batch_size
        end = min(start + self.batch_size, len(self.df))
        batch_indices = self.indices[start:end]

        X_batch = []
        y_batch = []

        # Load and preprocess each image in the batch
        for idx in batch_indices:
            img_path = self.df['img_path'].iloc[idx]
            label = self.df['label'].iloc[idx]

            # Preprocess image
            img = preprocess_image(img_path, target_size=self.target_size)
            X_batch.append(img)
            y_batch.append(label)

        # Convert lists to arrays
        X_batch = np.array(X_batch, dtype=np.float32)
        y_batch = to_categorical(y_batch, num_classes=self.num_classes)

        # Apply augmentation if enabled
        if self.augment:
            X_batch, y_batch = next(self.datagen.flow(X_batch, y_batch, batch_size=self.batch_size))

        return X_batch, y_batch

    def on_epoch_end(self):
        # Shuffle indices for each epoch
        if self.shuffle:
            np.random.shuffle(self.indices)
```

- **Rotation** : Randomly rotates the image up to ± 15 degrees, helping the model learn from different orientations.
- **Width and Height Shifts**: Moves the image left/right or up/down by up to 10% of its dimensions, making the model robust to positional variations.
- **Shear**: Distorts the image along an axis, which can simulate a 3D tilt or slant.

- **Zoom:** Scales the image up or down by 10%, making the model learn from both zoomed-in and zoomed-out views.
- **Horizontal Flip :** Flips the image horizontally, providing a mirror image and increasing variability.
- **Fill Mode:** Handles newly exposed pixels after a transformation by filling them with the nearest existing pixel value, maintaining the context of the image.

3) Model used:

Custom layer Cnn is being used for the classification model.

Splitting the data set into 90% and 10 % for training and validation

```
model=Sequential()
model.add(Conv2D(32,kernel_size=(3,3),padding='same',activation='relu',input_shape=(150,150,3)))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2),strides=2,padding='same'))

model.add(Conv2D(64,kernel_size=(3,3),padding='same',activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2),strides=2,padding='same'))

model.add(Conv2D(128,kernel_size=(3,3),padding='same',activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2,2),strides=2,padding='same'))

model.add(Flatten())
model.add(Dense(256,activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(128,activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(44,activation='softmax'))

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
```

First Convolutional Block:

- **Conv2D(32, ...):** The first convolutional layer uses 32 filters, each of size 3x3, with ' same ' padding to maintain the input size. The ReLU activation introduces non-linearity, allowing the model to learn complex patterns.
- **BatchNormalization() :** Normalizes the outputs of the convolutional layer, accelerating training and improving stability.
- **MaxPooling2D:** Reduces the spatial dimensions (down-sampling) to lower computational complexity and helps in extracting dominant features while maintaining important data.

Second Convolutional Block:

- **Conv2D(64, ...):** Increases the filter count to 64, enabling the model to learn more detailed and abstract features.
- **MaxPooling2D:** Further down-samples the feature maps, consolidating learned features.

Third Convolutional Block:

- **Conv2D(128, ...)**: With 128 filters, this layer allows the model to recognize complex, high-level patterns in the image.
- **MaxPooling2D**: Continues to reduce the size of the feature maps, preparing them for the dense layers.

Fully Connected Layers:

- **Flatten()**: Transforms the 3D feature maps into a 1D vector, which can be fed into dense layers.
- **Dense(256, activation='relu')** and **Dense(128, activation='relu')**: These dense layers act as the model's classifier, learning relationships between high-level features.
- **Dropout(0.2)**: Randomly deactivates 20% of the neurons during training to reduce overfitting and improve generalization.
- **Dense(44, activation='softmax')**: The output layer with 44 units (corresponding to the number of classes) uses the softmax activation to output class probabilities.

4)summary of the model():

```
[97]: model.summary()
```

Model: "sequential_7"

Layer (type)	Output Shape	Param #
conv2d_21 (Conv2D)	(None, 150, 150, 32)	896
batch_normalization_21 (Batch Normalization)	(None, 150, 150, 32)	128
max_pooling2d_21 (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_22 (Conv2D)	(None, 75, 75, 64)	18496
batch_normalization_22 (Batch Normalization)	(None, 75, 75, 64)	256
max_pooling2d_22 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_23 (Conv2D)	(None, 37, 37, 128)	73856
batch_normalization_23 (Batch Normalization)	(None, 37, 37, 128)	512
max_pooling2d_23 (MaxPooling2D)	(None, 18, 18, 128)	0
flatten_7 (Flatten)	(None, 41472)	0
dense_21 (Dense)	(None, 256)	10617088
dropout_14 (Dropout)	(None, 256)	0
dense_22 (Dense)	(None, 128)	32896
dropout_15 (Dropout)	(None, 128)	0
dense_23 (Dense)	(None, 44)	5676
Total params: 10,749,804		
Trainable params: 10,749,356		

5) accuracy and validation output:

The output after each epoch is given below

```
Epoch 1/10
282/282 [=====] - 142s 498ms/step - loss: 3.4085 - accuracy: 0.2579 - val_loss: 6.6813 - val_accuracy: 0.1460
Epoch 2/10
282/282 [=====] - 175s 618ms/step - loss: 2.6111 - accuracy: 0.3580 - val_loss: 2.7089 - val_accuracy: 0.3240
Epoch 3/10
282/282 [=====] - 146s 516ms/step - loss: 2.3086 - accuracy: 0.4189 - val_loss: 3.0124 - val_accuracy: 0.2200
Epoch 4/10
282/282 [=====] - 149s 529ms/step - loss: 2.0914 - accuracy: 0.4628 - val_loss: 2.1283 - val_accuracy: 0.4450
Epoch 5/10
282/282 [=====] - 146s 518ms/step - loss: 1.9635 - accuracy: 0.4906 - val_loss: 1.8201 - val_accuracy: 0.5430
Epoch 6/10
282/282 [=====] - 146s 517ms/step - loss: 1.8548 - accuracy: 0.5189 - val_loss: 3.0617 - val_accuracy: 0.2370
Epoch 7/10
282/282 [=====] - 147s 520ms/step - loss: 1.7537 - accuracy: 0.5381 - val_loss: 1.6381 - val_accuracy: 0.5770
Epoch 8/10
282/282 [=====] - 142s 505ms/step - loss: 1.6842 - accuracy: 0.5571 - val_loss: 2.2317 - val_accuracy: 0.4230
Epoch 9/10
282/282 [=====] - 134s 474ms/step - loss: 1.6060 - accuracy: 0.5756 - val_loss: 1.6029 - val_accuracy: 0.5640
Epoch 10/10
282/282 [=====] - 136s 482ms/step - loss: 1.5573 - accuracy: 0.5823 - val_loss: 1.9954 - val_accuracy: 0.4920
```

6) accuracy and f1 score:

```
rint(report)
```

	precision	recall	f1-score	support
0	0.97	0.37	0.53	79
1	0.68	0.33	0.44	40
2	0.43	0.67	0.52	9
3	0.00	0.00	0.00	8
4	0.00	0.00	0.00	4
5	0.00	0.00	0.00	6
6	0.00	0.00	0.00	3
7	0.55	0.64	0.59	28
8	0.00	0.00	0.00	10
9	0.05	0.06	0.05	17
10	0.00	0.00	0.00	7
12	0.74	0.85	0.79	92
13	0.00	0.00	0.00	9
14	0.17	0.44	0.24	16
15	0.06	0.07	0.07	14
16	0.20	0.41	0.27	27
17	0.00	0.00	0.00	4
18	0.98	0.64	0.77	74
19	0.52	0.64	0.57	86
20	0.00	0.00	0.00	11
21	0.20	0.14	0.17	7
22	0.66	0.70	0.68	80
23	0.00	0.00	0.00	5
24	0.33	0.93	0.48	46
25	0.00	0.00	0.00	10
26	0.00	0.00	0.00	11
27	0.00	0.00	0.00	9
28	1.00	0.31	0.48	32
29	0.00	0.00	0.00	12
30	1.00	0.70	0.82	73
31	0.00	0.00	0.00	5
32	0.00	0.00	0.00	2
33	0.14	0.85	0.25	20
34	1.00	0.38	0.56	26
35	0.00	0.00	0.00	6
36	0.00	0.00	0.00	5
37	0.55	0.55	0.55	11
38	1.00	0.38	0.55	32
39	0.00	0.00	0.00	1
40	0.31	0.39	0.34	31
41	0.00	0.00	0.00	2
42	0.00	0.00	0.00	3
43	0.35	0.30	0.32	27
accuracy			0.49	1000
macro avg	0.28	0.25	0.23	1000
weighted avg	0.58	0.49	0.49	1000